Ejercicio 2.7 Diseño

a) Realice la elección de un diseño para esta realidad.

Debe indicar los Tipos Abstractos de Datos elegidos, por ejemplo:

Vehículos = Secuencia(Vehículo)

Vehículo = Matricula (String) x Marca (String) x Año (int)

Auto = Matricula (String) x Marca (String) x Año (int) x CantidadPuertas (int) x PrecioBase (double)

Moto = Matricula (String) x Marca (String) x Año (int) x Cilindrada (int) x PrecioBase (double)

También debe incluir observaciones que justifiquen la elección realizada.

Observaciones:

- Elección de la Estructura (Secuencia): Se elige el TAD Secuencia para la colección global porque permite mutabilidad dinámica (altas y bajas en tiempo de ejecución) y provee una interfaz iterable indispensable para resolver linealmente los algoritmos de acumulación (sumas), búsqueda de extremos (máximos/mínimos) y filtrados.
- Modelado de Clase Abstracta: El TAD Vehículo se define como clase abstracta ya que en el dominio del negocio no existen "vehículos genéricos", sino entidades concretas (Auto o Moto). Esto impide la instanciación errónea de la clase base y promueve la reutilización de código.
- Polimorfismo Dinámico: El método precioAlquilerDiario() comparte firma conceptual en todo el catálogo pero su algoritmo varía según el tipo de vehículo. Se define abstracto en la clase padre para delegar su resolución al enlace dinámico en tiempo de ejecución a través de la sobreescritura en las subclases.
- Atributos Específicos: El atributo precioBase se mantiene en los TADs derivados Auto y Moto. Aunque comparten tipo de dato, pertenecen a naturalezas y ecuaciones de cálculo de negocio independientes para cada subtipo de vehículo, respetando el modelo conceptual.

b) Realice el correspondiente desglose de requerimientos.

Para cada requerimiento, indique:

- Entrada
- Salida
- Proceso

Debe desglosar al menos los siguientes casos: Requerimiento 1a: Alta

de vehículo.

Entrada: Objeto v de tipo Vehiculo (o subtipo Auto/Moto) Salida: Ninguna Proceso:

1. Recorrer la coleccion buscando si ya existe un elemento cuya matricula coincida con v.getMatricula().
2. SI la matricula ya esta registrada, ENTONCES lanzar Error: "La matricula ya existe".
3. SINO, invocar el metodo secuencia.agregar(v).

Requerimiento 1b: Baja de vehículo.

Entrada: matricula (String) del vehiculo a eliminar Salida: Ninguna Proceso:

1. Recorrer la coleccion buscando si existe un elemento con esa matricula.
2. SI no se encuentra ningun vehiculo, ENTONCES lanzar Error: "La matricula no existe".
3. SINO, invocar el metodo secuencia.eliminar(matricula).

Requerimiento 2: Calcular suma de precios de alquiler diario.

Entrada: Ninguna Salida: suma (double) Proceso:

1. Inicializar la variable suma = 0.0.
2. Obtener el iterador de la coleccion: iterador = secuencia.obtenerIterador().
3. MIENTRAS iterador.hasNext() (tenga un siguiente elemento): Asignar v = iterador.next(). Calcular suma = suma + v.precioAlquilerDiario().
4. RETORNAR suma.

Requerimiento 3: Obtener vehículos con mayor y menor precio de alquiler diario.

Entrada: Ninguna Salida: Datos del vehiculo con mayor precio y datos del vehiculo con menor precio (String) Proceso:

1. SI secuencia.esVacia() es verdadero, ENTONCES lanzar Error: "No hay vehiculos en el sistema".
2. Obtener el iterador de la coleccion: iterador = secuencia.obtenerIterador().
3. Tomar el primer elemento para inicializar: primerVehiculo = iterador.next().
4. Asignar mayor = primerVehiculo y menor = primerVehiculo.
5. MIENTRAS iterador.hasNext() (tenga un siguiente elemento): Asignar v = iterador.next(). SI v.precioAlquilerDiario() > mayor.precioAlquilerDiario(), ENTONCES mayor = v. SI v.precioAlquilerDiario() < menor.precioAlquilerDiario(), ENTONCES menor = v.
6. RETORNAR mayor.obtenerDatos() y menor.obtenerDatos().

Requerimiento 4: Contar motos por cilindrada.

Entrada: cilindradaBuscada (int) Salida: contador (int) Proceso:

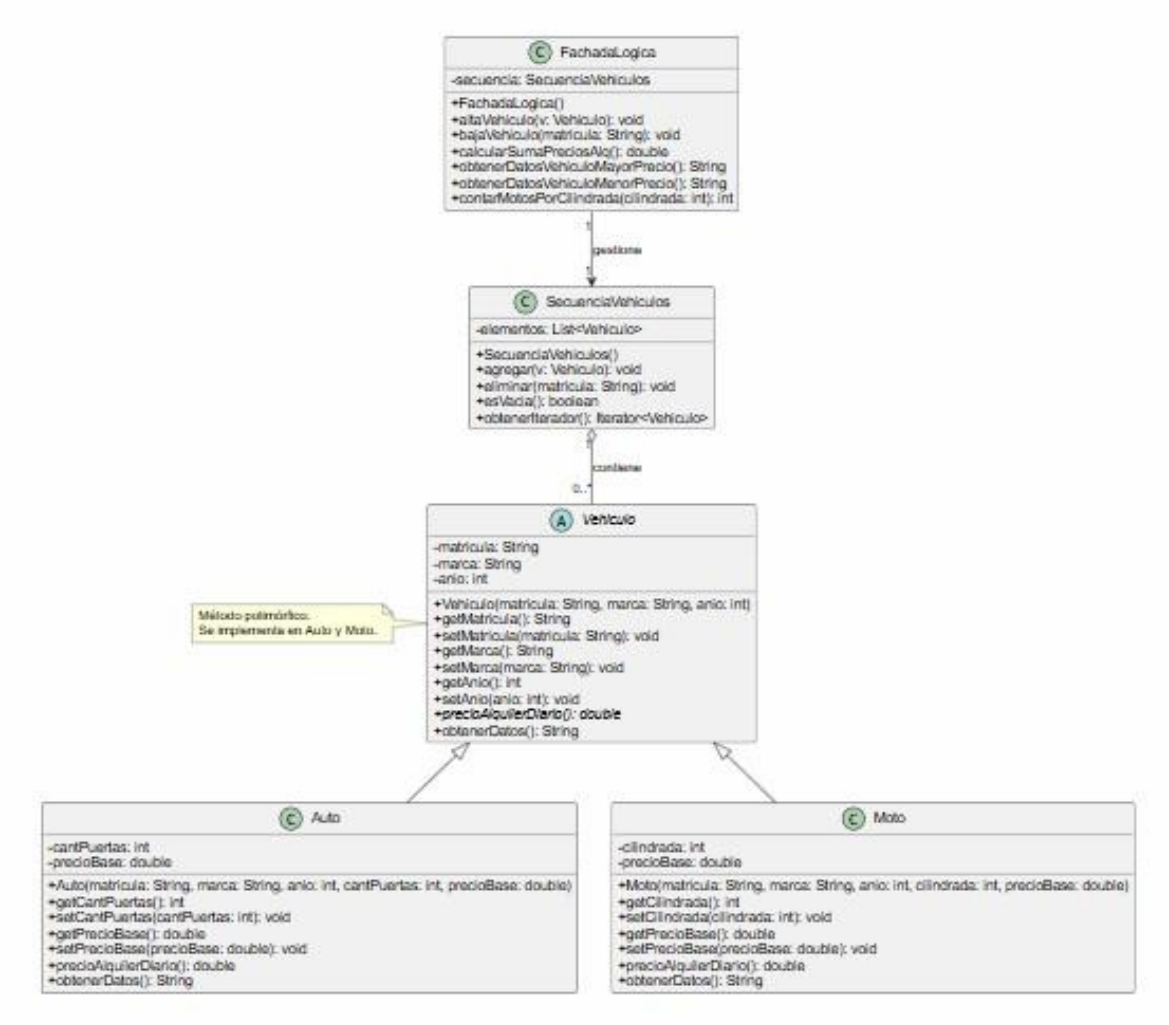
1. Inicializar la variable contador = 0.
2. Obtener el iterador de la coleccion: iterador = secuencia.obtenerIterador().
3. MIENTRAS iterador.hasNext() (tenga un siguiente elemento): Asignar v = iterador.next(). SI v es una instancia de la clase Moto, ENTONCES: Realizar casteo explicito: m = (Moto) v. SI m.getCilindrada() == cilindradaBuscada, ENTONCES contador = contador + 1.
4. RETORNAR contador.

C)En base a las partes anteriores, realice la especificación del diseño elegido.

Debe presentar el **Diagrama de Clases de Implementación en UML**, mostrando solamente los atributos, y luego detallar la especificación de cada clase por separado, incluyendo:

- atributos;
- constructor;
- selectores;
- modificadores necesarios;
- métodos específicos;

- métodos polimórficos, si corresponde.



Clase Abstracta Vehiculo Atributos:

- matricula: String
- marca: String
- anio: int Metodos:

- Vehiculo(matricula: String, marca: String, anio: int)
- getMatricula(): String
- setMatricula(matricula: String): void
- getMarca(): String
- setMarca(marca: String): void
- getAnio(): int
- setAnio(anio: int): void
- {abstract} precioAlquilerDiario(): double
- obtenerDatos(): String

Clase Auto (Hereda de Vehiculo) Atributos:

- cantPuertas: int
- precioBase: double Metodos:

- Auto(matricula: String, marca: String, anio: int, cantPuertas: int, precioBase: double)
- getCantPuertas(): int
- setCantPuertas(cantPuertas: int): void

- getPrecioBase(): double
- setPrecioBase(precioBase: double): void
- precioAlquilerDiario(): double
- obtenerDatos(): String

Clase Moto (Hereda de Vehiculo) Atributos:

- cilindrada: int
- precioBase: double Metodos:

- Moto(matricula: String, marca: String, anio: int, cilindrada: int, precioBase: double)
- getCilindrada(): int
- setCilindrada(cilindrada: int): void
- getPrecioBase(): double
- setPrecioBase(precioBase: double): void
- precioAlquilerDiario(): double
- obtenerDatos(): String

Clase SecuenciaVehiculos Atributos:

- elementos: List Metodos:

- SecuenciaVehiculos()
- agregar(v: Vehiculo): void
- eliminar(matricula: String): void
- esVacia(): boolean
- obtenerIterador(): Iterator

Clase FachadaLogica Atributos:

- secuencia: SecuenciaVehiculos Metodos:

- FachadaLogica()
- altaVehiculo(v: Vehiculo): void
- bajaVehiculo(matricula: String): void
- calcularSumaPreciosAlq(): double
- obtenerDatosVehiculoMayorPrecio(): String
- obtenerDatosVehiculoMenorPrecio(): String
- contarMotosPorCilindrada(cilindrada: int): int

Codigo PlantUML Corregido:

@startuml

skinparam classAttributeIconSize 0

abstract class Vehiculo {

matricula: String

marca: String

anio: int

Vehiculo(matricula: String, marca: String, anio: int)

getMatricula(): String

setMatricula(matricula: String): void

getMarca(): String

setMarca(marca: String): void

getAnio(): int

setAnio(anio: int): void

{abstract} precioAlquilerDiario(): double

obtenerDatos(): String

}

class Auto {

cantPuertas: int

precioBase: double

Auto(matricula: String, marca: String, anio: int, cantPuertas: int, precioBase: double)

getCantPuertas(): int

setCantPuertas(cantPuertas: int): void

getPrecioBase(): double

setPrecioBase(precioBase: double): void

precioAlquilerDiario(): double

obtenerDatos(): String

}

class Moto {

cilindrada: int

precioBase: double

Moto(matricula: String, marca: String, anio: int, cilindrada: int, precioBase: double)

getCilindrada(): int

setCilindrada(cilindrada: int): void

getPrecioBase(): double

setPrecioBase(precioBase: double): void

precioAlquilerDiario(): double

obtenerDatos(): String

}

class SecuenciaVehiculos {

elementos: List<Vehiculo>

SecuenciaVehiculos()

agregar(v: Vehiculo): void

eliminar(matricula: String): void

esVacia(): boolean

obtenerIterador(): Iterator<Vehiculo>

}

class FachadaLogica {

secuencia: SecuenciaVehiculos

FachadaLogica()

altaVehiculo(v: Vehiculo): void

bajaVehiculo(matricula: String): void

calcularSumaPreciosAlq(): double

obtenerDatosVehiculoMayorPrecio(): String

obtenerDatosVehiculoMenorPrecio(): String

contarMotosPorCilindrada(cilindrada: int): int

}

Vehiculo <|-- Auto

Vehiculo <|-- Moto

FachadaLogica "1" --> "1" SecuenciaVehiculos: gestiona

SecuenciaVehiculos "1" o-- "0..*" Vehiculo: contiene

@enduml